Minimum calls required before calculating failure rate.

Example:
minimumNumberOfCalls = 5

failureRateThreshold

Percentage of failures allowed before opening circuit.

Example:
failureRateThreshold = 50

If more than 50% requests fail → circuit opens.

waitDurationInOpenState

Time circuit stays OPEN before testing service again.

Example:
waitDurationInOpenState = 30s

permittedNumberOfCallsInHalfOpenState

Number of test calls allowed when the circuit is HALF_OPEN.

Example:
3 test requests allowed.

---
6. CIRCUIT BREAKER STATES
---

1. CLOSED

Normal operation.
Requests go to primary service.

2. OPEN

Service failure detected.
Primary service calls are blocked.
The fallback method is executed.

3. HALF_OPEN

After wait duration expires.
Limited calls are allowed to test service.

If successful → circuit closes.
If failure → circuit opens again.

---
7. REQUEST FLOW

CLIENT
 |
 v
CIRCUIT BREAKER
 |
 v
PRIMARY SERVICE (REDIS)
 |
 | success
 v
RESPONSE TO CLIENT

If failure occurs:

CLIENT
 |
 v
CIRCUIT BREAKER
 |
 v
PRIMARY SERVICE FAILS
 |
 v
FALLBACK METHOD
 |
 v
DATABASE RESPONSE

---
8. MONITORING CIRCUIT BREAKER

Check circuit breaker status using actuator:

http://localhost:8081/actuator/health

or

http://localhost:8081/actuator/circuitbreakers

---
9. KEY INTERVIEW POINT

Circuit Breaker prevents cascading failures in microservices by temporarily stopping calls to a failing service and redirecting requests to a fallback mechanism.

States of circuit breaker
==========

WHICH LOGIC RUNS IN WHICH STATE

STATE	PRIMARY LOGIC (REDIS)	FALLBACK
CLOSED	YES	Only if exception occurs
OPEN	NO	Always executed
HALF_OPEN	Limited requests	If those requests fail

==========

==========

SCENARIO 3: Notification Service Failure

1. Context

- After key actions in the audit system:
    - Audit assigned
    - Finding approved
    - Status changed
- Backend triggers notification service:
    - Email service

2. Problem

- Notification service is down / slow
- API call to notification fails
- Without protection:
    - Main business flow also fails
    - User action (approve/assign) gets blocked

3. Circuit Breaker Usage

- Wrap notification call with circuit breaker

Example:
@CircuitBreaker(name = "notificationService", fallbackMethod = "notificationFallback")

4. Behavior

- After multiple failures → circuit opens
- System stops calling notification service
- Immediately executes fallback

5. Fallback Strategy

- Store notification event in:
    - Retry table (DB)
    - OR internal queue
- Scheduler / batch job retries later

6. Outcome

- Business operation SUCCESS (audit continues)
- Notification becomes eventually consistent
- System avoids unnecessary failures

7. Key Point

- Non-critical services (like notifications) should NEVER break main flow

==========

SCENARIO 4: Rule Engine External Dependency Failure

1. Context

- Rule engine evaluates:
  - Dynamic audit rules
  - Conditions based on JSON config
- May depend on:
  - External config service
  - Remote API for enrichment

2. Problem

- External dependency is slow or unavailable
- Rule evaluation gets delayed or fails
- Can block:
  - Audit creation
  - Finding processing

3. Circuit Breaker Usage

- Wrap external calls inside rule engine

Example:
@CircuitBreaker(name = "ruleEngineService", fallbackMethod = "ruleFallback")

4. Behavior

- On repeated failures → circuit opens
- External calls are skipped
- Fallback logic is triggered immediately

5. Fallback Strategy

- Use:
    - Default rules
    - Cached rule configurations
- OR skip non-critical validations

6. Outcome

- Audit workflow continues without interruption
- Some rules may not apply (degraded mode)
- System remains available and stable

7. Key Point

- Critical flows should not depend fully on external rule evaluation
- Always design rule engine with fallback capability

Optimistic vs pesimistic lock

OPTIMISTIC LOCKING (JPA / HIBERNATE)

Definition
Optimistic locking is a concurrency control mechanism used to prevent lost updates when multiple transactions try to update the same database row simultaneously. It assumes conflicts are rare and detects them during update instead of locking the row.

How It Works
Optimistic locking uses a VERSION column in the table. Every time the row is updated, the version value is incremented. During update, Hibernate checks if the version in DB matches the version read earlier.

If version mismatch occurs → OptimisticLockException is thrown.

ENTITY EXAMPLE

```
@Entity
public class Student {

    @Id
    private Long id;

    private String name;

    private int marks;

    @Version
    private Integer version;
}
```

DATABASE TABLE

```
student
--------------------------------
id | name  | marks | version
--------------------------------
1  | Tejas | 80    | 1
```

INTERNAL SQL GENERATED BY HIBERNATE

```
UPDATE student
SET marks = ?, version = version + 1
WHERE id = ? AND version = ?
```

CONCURRENT UPDATE SCENARIO

Step 1
Thread A reads version = 1
Thread B reads version = 1

Step 2
Thread A updates record
version becomes 2

Step 3
Thread B tries update with version = 1

UPDATE student
SET marks = 90
WHERE id = 1 AND version = 1

No rows updated → Hibernate throws OptimisticLockException

JDBC LEVEL INTERNAL FLOW

executeUpdate()
↓
if rowsUpdated == 0
↓
OptimisticLockException

KEY CHARACTERISTICS

• Uses VERSION column in database
• No row-level database locking
• Multiple reads allowed
• Conflict detected during update
• Prevents lost update problem
• Improves performance in high-read systems

WHEN TO USE

• Read-heavy applications
• Systems with low conflict probability
• Microservices and enterprise applications
• Audit systems, order systems, inventory updates

INTERVIEW ONE-LINE ANSWER

Optimistic locking prevents concurrent update conflicts using a version column. During update, Hibernate checks the version value in the WHERE clause, and if the version has changed since it was read, an OptimisticLockException is thrown.

PESSIMISTIC LOCKING (JPA / DATABASE)

Definition
Pessimistic locking is a concurrency control mechanism where the database row is locked when it is read so that other transactions cannot modify it until the current transaction completes.

It assumes conflicts are likely, so the system locks the data immediately.

WHERE WE APPLY IT

In Spring Boot / JPA it is applied in the repository layer using @Lock annotation and works inside a transaction.

Example Repository

```
@Repository
public interface StudentRepository extends JpaRepository<Student, Long> {

    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("select s from Student s where s.id = :id")
    Student findStudentForUpdate(Long id);
}
```

SERVICE LAYER

```
@Transactional
public void updateMarks(Long id){

    Student s = repo.findStudentForUpdate(id);

    s.setMarks(90);
}
```

Transaction Flow

Transaction start

↓
SELECT query with lock
↓
Row gets locked
↓
Update happens
↓
Transaction commit
↓
Lock released

SQL GENERATED BY DATABASE

SELECT * FROM student WHERE id = ? FOR UPDATE

This locks the selected row until the transaction finishes.

ROW LEVEL LOCKING

Pessimistic locking locks only the selected row, not the entire table.

Example

Thread A locks row id=1
Thread B can still update row id=2

TYPES OF PESSIMISTIC LOCK

1. PESSIMISTIC_READ

@Lock(LockModeType.PESSIMISTIC_READ)

Behavior
• Other transactions can read the row
• Updates are blocked until transaction completes

2. PESSIMISTIC_WRITE

@Lock(LockModeType.PESSIMISTIC_WRITE)

Behavior

• Prevents other transactions from updating the row
• Other transactions trying to lock or update must wait

3. PESSIMISTIC_FORCE_INCREMENT

• Works with versioned entities
• Locks the row and increments version

WHEN TO USE

• Banking transactions
• Payment processing
• Inventory management
• Ticket booking systems
• Critical data updates

KEY POINTS

• Implemented using database row locks
• Uses SQL "SELECT ... FOR UPDATE"
• Works inside transactions
• Prevents concurrent updates
• Can lead to deadlocks if not handled properly

Exceptions

Exception	Type	Cause (When it happens)
SQLException	Checked (`Exception → SQLException`)	Database connection failure, wrong SQL query, DB down
IOException	Checked (`Exception → IOException`)	File read/write failure, network stream issue
FileNotFoundException	Checked (`IOException → FileNotFoundException`)	File path incorrect or file missing
ParseException	Checked (`Exception → ParseException`)	Date or string parsing fails
InterruptedException	Checked (`Exception → InterruptedException`)	Thread interrupted during sleep/wait
ClassNotFoundException	Checked (`Exception → ClassNotFoundException`)	Class loader cannot find a class
MalformedURLException	Checked (`IOException → MalformedURLException`)	Invalid URL format
InstantiationException	Checked (`Exception → InstantiationException`)	Object creation using reflection fails
NoSuchMethodException	Checked (`Exception → NoSuchMethodException`)	Method not found during reflection
InvocationTargetException	Checked (`Exception → InvocationTargetException`)	Exception thrown during reflection method invocation

Exception	Type	Cause (When it happens)
NullPointerException	Unchecked (`RuntimeException → NullPointerException`)	Calling method on null object
IllegalArgumentException	Unchecked (`RuntimeException → IllegalArgumentException`)	Passing invalid parameter to method
NumberFormatException	Unchecked (`IllegalArgumentException → NumberFormatException`)	Converting invalid string to number
IndexOutOfBoundsException	Unchecked (`RuntimeException → IndexOutOfBoundsException`)	Accessing invalid index in array/list
ArithmeticException	Unchecked (`RuntimeException → ArithmeticException`)	Division by zero
ClassCastException	Unchecked (`RuntimeException → ClassCastException`)	Casting object to incompatible type
IllegalStateException	Unchecked (`RuntimeException → IllegalStateException`)	Method called at wrong time/state
UnsupportedOperationException	Unchecked (`RuntimeException → UnsupportedOperationException`)	Operation not supported
ConcurrentModificationException	Unchecked (`RuntimeException → ConcurrentModificationException`)	Modifying collection during iteration
ArrayStoreException	Unchecked (`RuntimeException → ArrayStoreException`)	Wrong type stored in array

transactional DeadLock

Deadlock in Transaction

Deadlock occurs when two or more transactions hold locks on resources and each transaction waits for the other to release the lock. Because of this, none of the transactions can proceed and the database detects a deadlock and aborts one transaction.

Common Causes of Deadlock

1. Updating rows in different order
   Example:
   Transaction A updates row 1 then row 2
   Transaction B updates row 2 then row 1
   Both transactions wait for each other's lock.

2. Long running transactions
   If a transaction holds a lock for a long time (external API call, heavy processing), other transactions may wait and cause deadlock.

3. Pessimistic locking
   Using SELECT ... FOR UPDATE or JPA PESSIMISTIC_WRITE can create lock conflicts between concurrent transactions.

4. Missing database indexes
   Without indexes, database may scan and lock many rows or even table level locks which increases deadlock chances.

5. Bulk updates or deletes
   Large updates inside a transaction can lock many rows.

6. Foreign key cascading operations
   Updating parent and child tables in different order by different transactions can lead to deadlock.

How to Avoid Deadlock

1. Access tables and rows in the same order in all transactions.

2. Keep transactions short.
   Do not call external APIs or long processing inside transactions.

3. Use proper indexes to avoid locking large number of rows.

4. Prefer optimistic locking when possible.
   Example:
   @Version
   private Long version;

5. Avoid unnecessary pessimistic locks.

6. Implement retry mechanism if deadlock occurs.

Common Exception in Spring

org.springframework.dao.DeadlockLoserDataAccessException

Sonarqube

==========
SONARQUBE – COMPLETE NOTES (REAL PROJECT)
==========

1) WHAT IS SONARQUBE

SonarQube is a Static Code Analysis tool used to continuously inspect code quality, security vulnerabilities, bugs, and maintainability issues.

It scans the source code without executing the application.

Goal:
Improve code quality and prevent bad code from reaching production.

Main capabilities:

• Detect Bugs
• Detect Security Vulnerabilities
• Detect Code Smells
• Measure Code Coverage
• Detect Code Duplication
• Calculate Technical Debt
• Enforce Quality Gates
• Integrate with CI/CD pipelines

Example technologies supported:

Java
Spring Boot
Python
JavaScript
TypeScript
C#
C++
Go

---

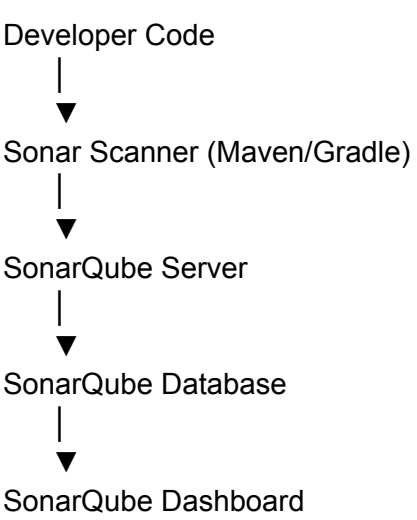
2) SONARQUBE ARCHITECTURE (REAL APPLICATION)

Main components:

1) SonarQube Server

2) Sonar Scanner
3) SonarQube Database
4) Web Dashboard

Architecture Flow



Components explanation

Sonar Scanner

Tool that scans the source code.

Examples:
sonar-scanner
mvn sonar:sonar
gradle sonarqube

SonarQube Server

Processes analysis results and applies rules.

SonarQube Database

Stores:

• issues
• metrics
• coverage
• technical debt
• history

Dashboard

Web UI used by developers to see issues.

---

3) HOW SONARQUBE IS CONFIGURED IN REAL PROJECT

Step 1 – Install SonarQube Server

Download SonarQube from

https://www.sonarqube.org/

Run server:

bin/windows-x86-64/StartSonar.bat

Server runs on:

http://localhost:9000

Default login

username: admin
password: admin

Step 2 – Install Sonar Scanner

Download:

https://docs.sonarsource.com/sonarqube/latest/analyzing-source-code/scanners/sonarscanner/

Add to PATH

Step 3 – Configure project (Maven Example)

pom.xml

```
<plugin>
  <groupId>org.sonarsource.scanner.maven</groupId>
  <artifactId>sonar-maven-plugin</artifactId>
  <version>3.9.1</version>
</plugin>
```

Step 4 – Create sonar-project.properties

```
sonar.projectKey=my-project
sonar.projectName=MyProject
sonar.projectVersion=1.0

sonar.sources=src/main/java
sonar.tests=src/test/java

sonar.java.binaries=target/classes

sonar.host.url=http://localhost:9000
sonar.login=SONAR_TOKEN
```

Step 5 – Run analysis

```
mvn clean verify sonar:sonar
```

OR

```
sonar-scanner
```

After execution results appear in SonarQube dashboard.

---

4) TYPES OF ISSUES DETECTED BY SONARQUBE

SonarQube categorizes issues into 4 types.

1) BUG

Definition

Code that may cause runtime failure.

Example

```
String name = null;
System.out.println(name.length());
```

Issue detected

Possible NullPointerException

---

2) VULNERABILITY (Security Issues)

Definition

Security weakness in code.

Example

```
String query =
"SELECT * FROM users WHERE id=" + userId;
```

Issue

SQL Injection

Fix

Use PreparedStatement

---

3) CODE SMELL

Definition

Bad coding practice that affects maintainability.

Example

```
if(isValid == true)
```

Better

```
if(isValid)
```

Another example

```
public void method() {

    if(a==1) {
        if(b==2) {
            if(c==3) {
                if(d==4) {
                }
            }
        }
    }

}
```

Issue

High Cyclomatic Complexity

---

4) SECURITY HOTSPOTS

Definition

Code that may be risky and requires manual review.

Example

```
File file = new File(userInputPath);
```

Risk

Path Traversal

---

5) METRICS SHOWN IN SONARQUBE DASHBOARD

Main metrics:

Bugs

Real defects.

Vulnerabilities

Security problems.

Code Smells

Maintainability issues.

Coverage

Unit test coverage.

Duplication

Repeated code blocks.

Technical Debt

Estimated time required to fix issues.

Example dashboard

Bugs: 2
Vulnerabilities: 1
Code Smells: 15
Coverage: 68%
Duplications: 4%

---

6) CODE COVERAGE IN SONARQUBE

SonarQube itself does NOT generate coverage.

It reads coverage reports from tools like:

Jacoco
Cobertura

Example Jacoco integration

```
<plugin>
   <groupId>org.jacoco</groupId>
   <artifactId>jacoco-maven-plugin</artifactId>
</plugin>
```